# CUET PG

# DAILY CLASS NOTES

## Computer Science

### Digital Fundamental

### Lecture – 14

### Flip-Flop

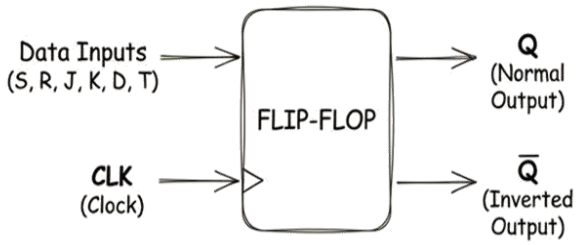


# Flip-Flop

## 0.1 What is a Flip-Flop?

**Definition:** A **Flip-Flop** is a fundamental building block in digital electronics. It is a circuit that has two stable states and can be used to store one bit (0 or 1) of state information. It is the basic unit of memory.

Architects Daughter

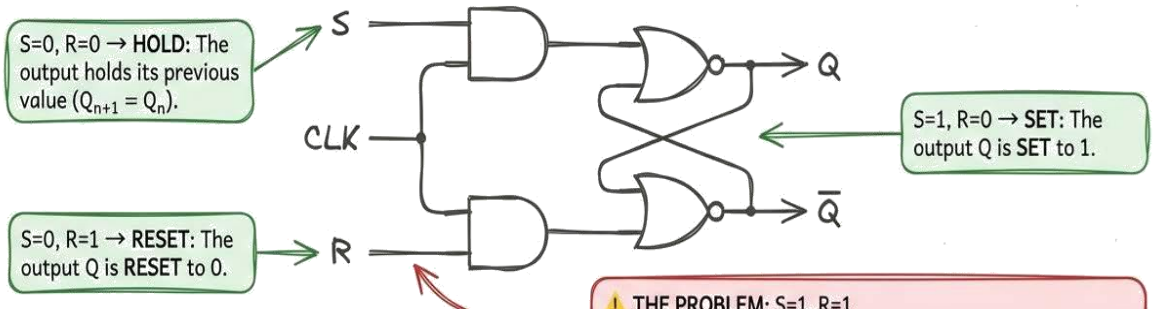



**Key Characteristics:**

- **Sequential Circuit**: Its output depends on both current inputs and the previous output (state).
- **Bistable**: It has two stable states (0 and 1).
- **1-Bit Memory**: It can store a single binary digit.
- **Clocked**: The output changes state only on a clock signal. This distinguishes it from a Latch.

## 1.0 The SR Flip-Flop (Set-Reset)

**Definition:** The SR Flip-Flop is the most basic type. It uses two inputs, **S (Set)** and **R (Reset)**, to control the stored state.




**Summary: Characteristic Table & Equation**

| S | R | Qn+1 |
|---|---|---|
| 0 | 0 | Qn (Hold) |
| 0 | 1 | 0 (Reset) |
| 1 | 0 | 1 (Set) |
| 1 | 1 | X (Invalid) |

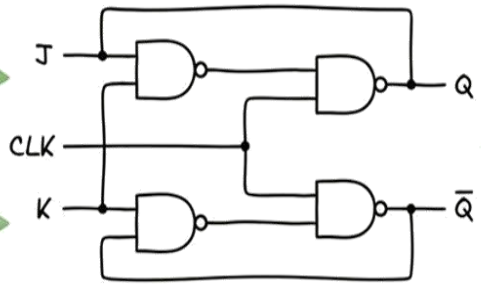
**Equation:** $Q_{n+1} = S + \overline{R}Q_n$ (with the condition SR=0)

# 2.0 The JK Flip-Flop (The Universal Flip-Flop)

**Opening Note:** The JK Flip-Flop improves on the SR by eliminating the invalid state. The inputs **J** and **K** act like S and R, but with a crucial difference. All other flip-flops can be designed using the JK.

**J=0, K=0 → HOLD**
(Same as SR)

**J=0, K=1 → RESET (Q=0)**
(Same as SR)




**J=1, K=0 → SET (Q=1)**
(Same as SR)

✅ **THE SOLUTION: J=1, K=1**
Instead of an invalid state, the output **TOGGLES**. It flips to the opposite of its **current state ($Q_{n+1} = \overline{Q}_n$)**. This is extremely useful for building counters.



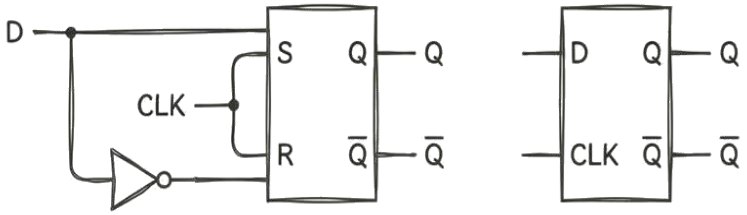

**Summary:** Characteristic Table & Equation

| J | K | $Q_{n+1}$ |
|---|---|---|
| 0 | 0 | $Q_n$ (Hold) |
| 0 | 1 | 0 (Reset) |
| 1 | 0 | 1 (Set) |
| 1 | 1 | $\overline{Q}_n$ (Toggle) |

**Equation:** $Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$

# 3.0 The D Flip-Flop (Data or Delay)

**Definition:** The D Flip-Flop has only one data input, **D**. Its purpose is to capture and store whatever value is on the D line when the clock ticks. It's a typical storage device.




**How it Works:** It's incredibly simple. The output ($Q_{n+1}$) is the same as the input (D).
- If **D = 0**, the flip-flop is **RESET** and Q becomes 0.
- If **D = 1**, the flip-flop is **SET** and Q becomes 1.

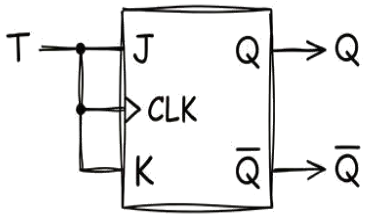
**Summary:** Characteristic Table & Equation

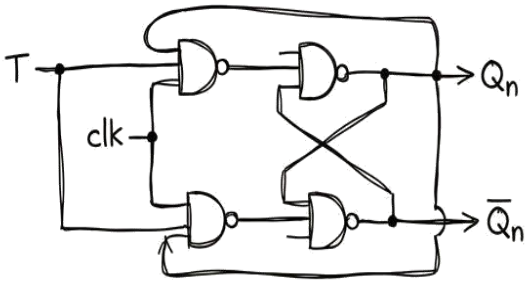
| D | $Q_{n+1}$ |
|---|---|
| 0 | 0 |
| 1 | 1 |

**Equation:** $Q_{n+1} = D$

## 4.0 The T Flip-Flop (Toggle)

**Definition:** The T Flip-Flop also has a single input, **T** (Toggle). Its specific purpose is to toggle the output state whenever the T input is high and the clock ticks.




**How it Works:** It has two simple modes based on the T input.

- If **T = 0**, the flip-flop **HOLDS** its current state ($Q_{n+1} = Q_n$).
- If **T = 1**, the flip-flop **TOGGLES** its state ($Q_{n+1} = \overline{Q}_n$).

**Summary:** Characteristic Table & Equation

| T | $Q_{n+1}$ |
|---|---|
| 0 | $Q_n$ (Hold) |
| 1 | $\overline{Q}_n$ (Toggle) |

**Equation:** $Q_{n+1} = T \oplus Q_n = T\overline{Q}_n + \overline{T}Q_n$

## Flip-Flops: At a Glance

| Flip-Flop | Characteristic Table | Characteristic Equation | Key Use Case |
|---|---|---|---|
| SR | S R \| $Q_{n+1}$<br>0 0 \| $Q_n$<br>0 1 \| 0<br>1 0 \| 1<br>1 1 \| X | $Q_{n+1} = S + \overline{R}Q_n$ | Basic memory, control circuits. |
| JK | J K \| $Q_{n+1}$<br>0 0 \| $Q_n$<br>0 1 \| 0<br>1 0 \| 1<br>1 1 \| $\overline{Q}_n$ | $Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$ | Universal flip-flop, counters. |
| D | D \| $Q_{n+1}$<br>0 \| 0<br>1 \| 1 | $Q_{n+1} = D$ | Data storage, shift registers. |
| T | T \| $Q_{n+1}$<br>0 \| $Q_n$<br>1 \| $\overline{Q}_n$ | $Q_{n+1} = T \oplus Q_n$ | Binary counters, frequency dividers. |

# End of Notes

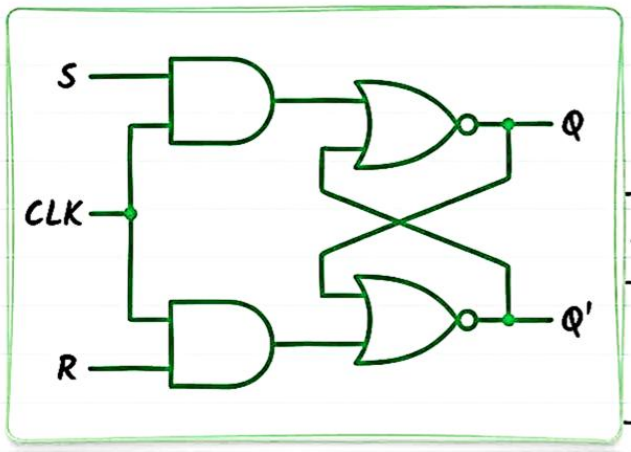
You've covered the core concepts of SR, JK, D, and T flip-flops.

Review the summary table and practice drawing the circuits!

**Keep Studying & Good Luck!**

# फ्लिप-फ्लॉप

## 1. SR फ्लिप-फ्लॉप (NOR Gate से)




**परिभाषा**

SR फ्लिप-फ्लॉप एक 1-बिट मेमोरी सेल है, मतलब यह एक बिट डेटा स्टोर कर सकता है। इसके दो इनपुट हैं - S (Set) और R (Reset)।

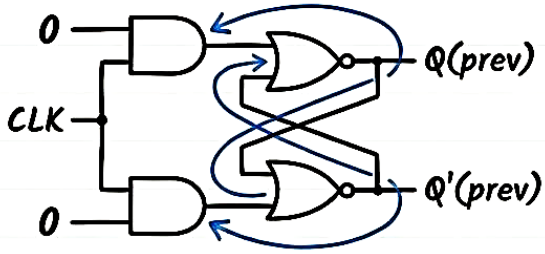
**CLK (Clock):** यह एक control signal है जो फ्लिप-फ्लॉप को बताता है कि इनपुट के आधार पर state कब बदलना है।

**S (Set):** जब S=1 होता है, तो यह आउटपुट Q को 1 पर 'सेट' कर देता है।

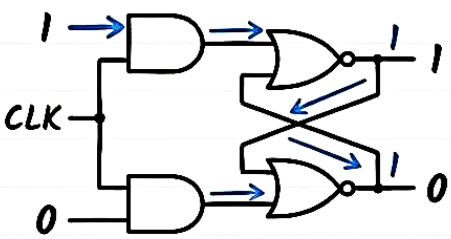
**R (Reset):** जब R=1 होता है, तो यह आउटपुट Q को 0 पर 'रीसेट' कर देता है।

## 2. SR फ्लिप-फ्लॉप - यह कैसे काम करता है?

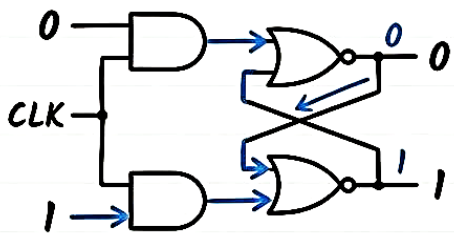
S=0, R=0 (Hold State)



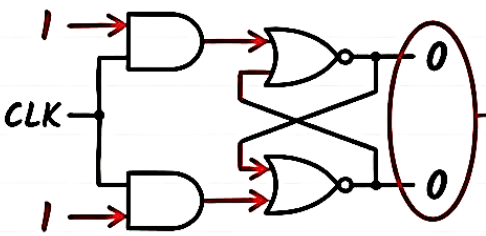

S=1, R=0 (Set State)




S=0, R=1 (Reset State)




S=1, R=1 (Invalid State)



## 3. SR फ्लिप-फ्लॉप: Truth Table

| CLK | S | R | $Q_{n+1}$ (Next State) | Operation |
|---|---|---|---|---|
| 0 | X | X | $Q_n$ | No Change |
| 1 | 0 | 0 | $Q_n$ | Hold |
| 1 | 0 | 1 | 0 | Reset |
| 1 | 1 | 0 | 1 | Set |
| 1 | 1 | 1 | ? | Invalid |

### समस्या: Invalid State

जब S और R दोनों 1 होते हैं, तो Q और Q' दोनों 0 हो जाते हैं। यह एक 'अमान्य' या 'वर्जित' (forbidden) स्थिति है क्योंकि Q' को हमेशा Q का पूरक (complement) होना चाहिए।

यह SR फ्लिप-फ्लॉप की सबसे बड़ी कमी है।

## 4. SR फ्लिप-फ्लॉप: Characteristic Equation

### Characteristic Table

| S | R | $Q_n$ | $Q_{n+1}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 (Hold) |
| 0 | 0 | 1 | 1 (Hold) |
| 0 | 1 | 0 | 0 (Reset) |
| 0 | 1 | 1 | 0 (Reset) |
| 1 | 0 | 0 | 1 (Set) |
| 1 | 0 | 1 | 1 (Set) |
| 1 | 1 | 0 | X (Invalid) |
| 1 | 1 | 1 | X (Invalid) |

### K-Map Simplification

| $R\ Q_n$ \ S | 0 | 1 | |
|---|---|---|---|
| 00 | 0 | 1 | → S |
| 01 | 0 | 0 | |
| 11 | 1 | 1 | → R'$Q_n$ |
| 10 | X | X | |

### Characteristic Equation

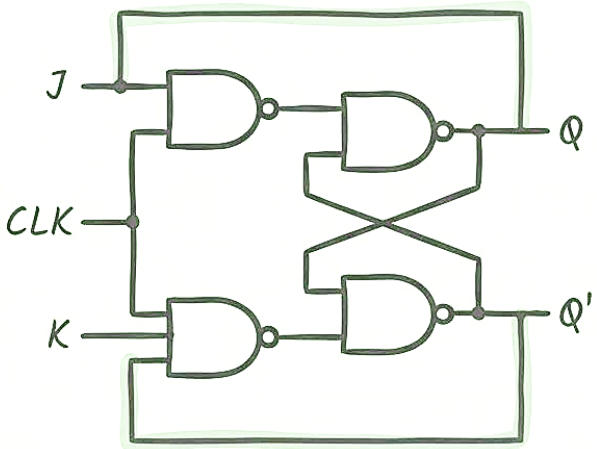
$$Q_{n+1} = S + R'Q_n$$

(यह समीकरण तभी मान्य है जब SR = 0)

## 5. JK फ्लिप-फ्लॉप (SR की समस्या का समाधान)

**परिभाषा**

*JK* फ्लिप-फ्लॉप, *SR* फ्लिप-फ्लॉप का एक परिष्कृत *(refined)* रूप है। यह *SR* फ्लिप-फ्लॉप की *Invalid State (S=1, R=1)* की समस्या को दूर करता है।

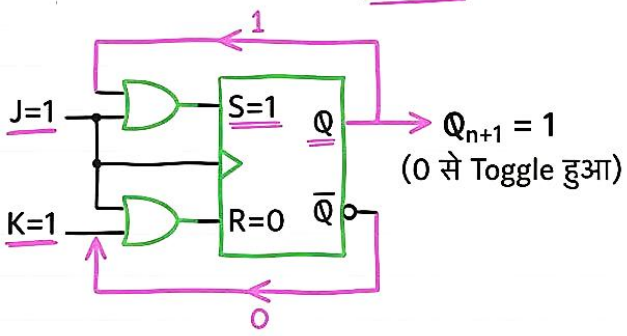



- *J* इनपुट, *S (Set)* की तरह काम करता है।
- *K* इनपुट, *R (Reset)* की तरह काम करता है।
- ***The Key Improvement:*** जब *J=1* और *K=1* होता है, तो *Invalid State* की जगह एक नई उपयोगी *state* आती है, जिसे *'Toggle'* कहते हैं।

## 6. JK फ्लिप-फ्लॉप का जादू: Toggle State

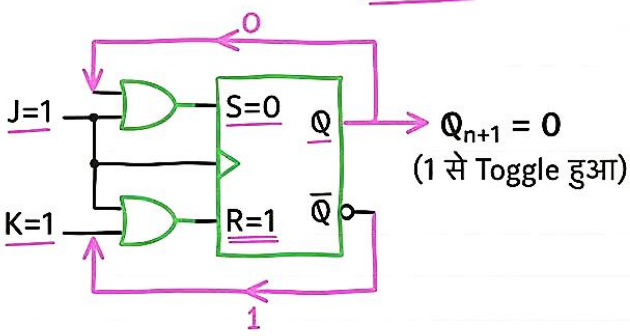
**जब J=1 और K=1 होता है तो क्या होता है?**

मान लीजिए $Q_n = 0$, $Q'_n = 1$




$Q_{n+1} = 1$
(0 से Toggle हुआ)

मान लीजिए $Q_n = 1$, $Q'_n = 0$




$Q_{n+1} = 0$
(1 से Toggle हुआ)

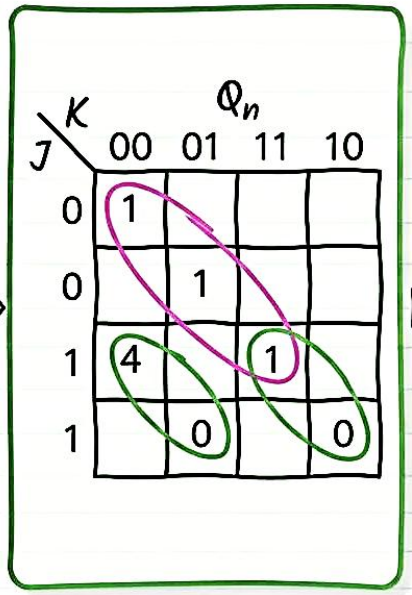
**निष्कर्ष**

J=1 और K=1 पर, आउटपुट हमेशा अपनी पिछली स्थिति का पूरक (complement) हो जाता है। इसी को Toggle करना कहते हैं।

## 7. JK फ्लिप-फ्लॉप: Table & Equation

**Characteristic Table**

| J | K | $Q_n$ | $Q_{n+1}$ | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | Hold |
| 0 | 0 | 1 | 1 | Hold |
| 0 | 1 | 0 | 0 | Reset |
| 0 | 1 | 1 | 0 | Reset |
| 1 | 0 | 0 | 1 | Set |
| 1 | 0 | 1 | 1 | Set |
| 1 | 1 | 0 | 1 | Toggle |
| 1 | 1 | 1 | 0 | Toggle |



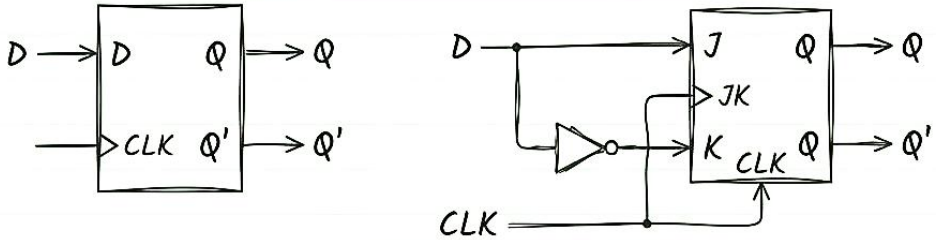

**Characteristic Equation**

$$Q_{n+1} = JQ_n' + K'Q_n$$

## 8. D फ्लिप-फ्लॉप (Data Flip-Flop)

**परिभाषा**

D फ्लिप-फ्लॉप को 'Data' या 'Delay' फ्लिप-फ्लॉप भी कहा जाता है। इसका मुख्य काम डेटा को एक क्लॉक साइकिल के लिए स्टोर या 'delay' करना है।




**Characteristic Equation** $Q_{n+1} = D$

**Exam Tip** इसका आउटपुट हमेशा इनपुट D के बराबर होता है (अगली क्लॉक पल्स पर)। यह Registers और Shift Registers में उपयोग होता है।

## 9. T फ्लिप-फ्लॉप (Toggle Flip-Flop)

**परिभाषा**

*T* फ्लिप-फ्लॉप का मतलब '*Toggle*' फ्लिप-फ्लॉप है। जब *T* इनपुट *1* होता है, तो इसका आउटपुट हर क्लॉक पल्स पर अपनी स्थिति बदलता (*toggle*) है।




✓ जब *T=0*, आउटपुट अपनी पुरानी *state* (*Qn*) बनाए रखता है। (*Hold*)
✓ जब *T=1*, आउटपुट *toggle* हो जाता है (*Qn'*)।

## 10. T फ्लिप-फ्लॉप: Table & Equation

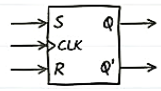
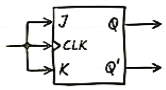
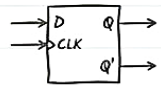
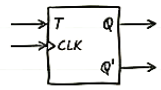
| T | $Q_n$ | $Q_{n+1}$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Characteristic Equation**

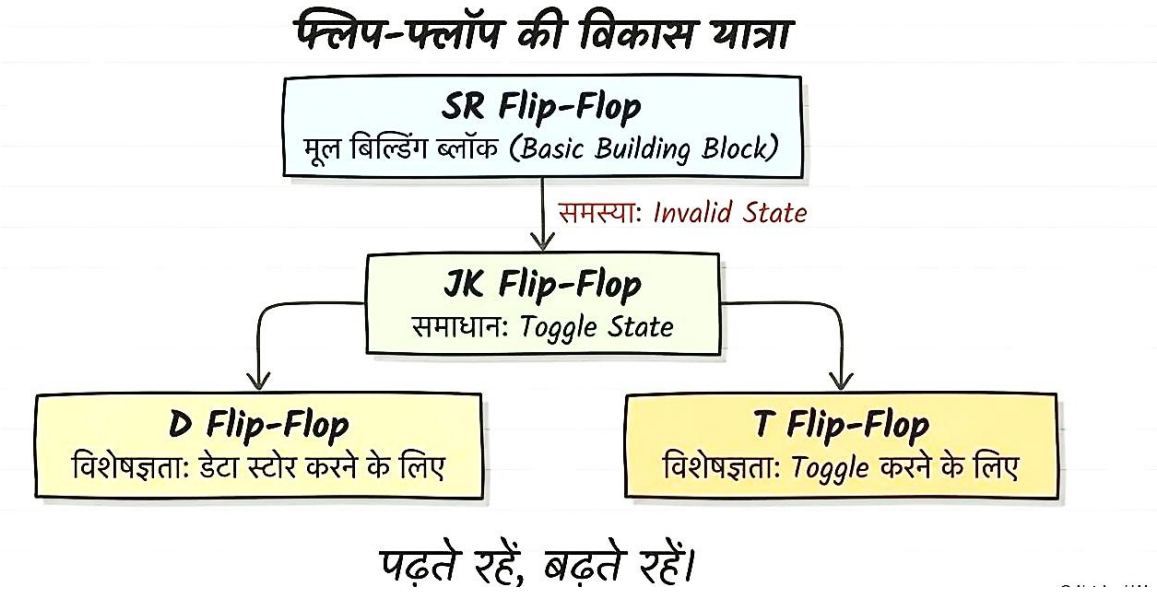
$$Q_{n+1} = T \oplus Q_n = T'Q_n + TQ_n'$$

**Exam Tip:** *T* क्लिप-फ्लॉप का मुख्य उपयोग *Frequency Divider Circuits* और बाइनरी काउंटर्स (*Binary Counters*) बनाने में होता है।

# 11. सभी फ्लिप-फ्लॉप का सारांश

| फ्लिप-फ्लॉप | इनपुट (Inputs) | Characteristic Equation ($Q_{n+1}$) | मुख्य विशेषता (Key Feature) |
|---|---|---|---|
|  SR | S, R | $Q_{n+1} = S + R'Q_n$ | Set/Reset, लेकिन S=1,R=1 पर Invalid State. |
|  JK | J, K | $Q_{n+1} = JQ_n' + K'Q_n$ | Set/Reset/Toggle, Invalid State की समस्या को हल करता है। |
|  D | D | $Q_{n+1} = D$ | Data Storage/Delay, Registers के लिए उपयोगी। |
|  T | T | $Q_{n+1} = T \oplus Q_n$ | Hold/Toggle, Counters और Frequency Division के लिए उपयोगी। |

# 12. मुख्य बातें




पढ़ते रहें, बढ़ते रहें।



PW Web/App: https://smart.link/7wwosivoicgd4

Library- https://smart.link/sdfez8ejd80if